



Norwegian University of
Science and Technology

Formally Verified Reduction Proof for the Half-Gates Garbling Scheme

Prateek Sharma

Submission date: April 2026
Main supervisor: Tjerand Silde, NTNU
Co-supervisors: Russel W. F. Lai and
Chris Brzuska, Aalto University

Norwegian University of Science and Technology
Department of Information Security and Communication Technology

Problem description

Garbled circuits are a fundamental cryptographic primitive used in secure two-party computation (2PC), enabling two parties to jointly compute a function without revealing their private inputs—only the output is revealed. The concept was introduced by Andrew Yao in 1986 [Yao86], where functions are expressed as Boolean circuits and each wire is labelled with encrypted wire keys. Early constructions required four ciphertexts per gate, resulting in significant communication overhead and limiting practical adoption.

Subsequent optimizations such as Point-and-Permute and the Half-Gates technique greatly reduced the number of ciphertexts needed per gate, leading to substantial performance improvements [BMR90; ZRE15]. These optimizations form the basis of many state-of-the-art secure computation frameworks.

Formal verification of cryptographic constructions is essential to ensure correctness and prevent subtle vulnerabilities arising from unchecked corner cases. Brzuska and Oechsner [BO23] developed a state-separating pen-and-paper reduction proof for Yao’s original garbling scheme. State-separating proofs model protocol components as isolated “packages” with private state and oracle interfaces, allowing for structured security reductions. Their work established security by reducing the garbling scheme to the IND-CPA security of the encryption primitive used for gate tables.

Domino is an automated verification tool specifically designed for state-separating proofs. It has been used to reason about complex real-world protocols such as TLS [Raj25] and WPA2 [BDE+22]. Domino offers a high level of automation and readability because proof scripts resemble the pseudocode found in traditional cryptographic proofs.

Furthermore, the above mentioned proof for Yao’s garbling scheme by Brzuska and Oechsner [BO23] was later formalized in Domino [BW]. It can serve as a good starting point for the thesis to observe how games may be converted from pen and paper proofs to Domino code.

The goal of this thesis is to formalize and mechanize a security reduction for the Half-Gates garbling scheme within the Domino framework. This includes:

- Developing a state-separating, pen-and-paper reduction proof for the Half-Gates technique.
- Encoding this proof in Domino.

This work extends formal verification techniques to an optimized and widely deployed garbling construction and demonstrates Domino’s applicability to modern cryptographic primitives.

Approved: yyyy-mm-dd – Tjerand Silde, NTNU (Main supervisor)

Abstract

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

This is the second paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

And after the second paragraph follows the third paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

After this fourth paragraph, we start a new paragraph sequence. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of

the original language. There is no need for special content, but the length of words should match the language.

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Preface

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Contents

List of Figures	vii
List of Tables	ix
List of Algorithms	xi
1 Example	1
1.1 First section (middling version)(ToC)	2
1.1.1 First subsection with some <i>Math</i> symbol	2
1.1.2 Mathematics	2
1.1.3 Source code example	3
2 Introduction	5
3 Background	7
3.1 Multi-Party Computation (MPC)	7
3.2 Garbling Schemes	7
3.2.1 Oblivious Transfer Protocols	7
3.3 Optimizations to Yao's Garbling scheme	7
3.3.1 Point and Permute	7
3.3.2 Free XOR	7
3.3.3 Garbled Row Reduction - 3	7
3.3.4 Garbled Row Reduction - 2	8
3.3.5 FleXOR	8
3.3.6 Half-Gates	8
3.3.7 Garbled Gadgets	8
3.4 Formal Verification and Tools	8
3.4.1 Easycrypt	8
3.4.2 Domino	8
4 Pen-and-Paper State-Separating Proof	9
4.1 State Separating Proofs	9

4.2	Existing Half-Gates Proof and how it is to be modelled into state separating proofs	9
5	Domino	11
5.1	What is Domino?	11
5.2	Using Domino	11
5.3	Coding Half-gates in Domino	11
5.4	Improvements	11
6	Conclusion	13
	References	15

List of Figures

1.1	A figure	2
-----	--------------------	---

List of Tables

1.1	A table	3
-----	-------------------	---

List of Algorithms

1.1	The Hello World! program in Java.	3
-----	---	---

Chapter 1

Example

Here is an example of how to use acronyms such as Norwegian University of Science and Technology (NTNU). The second time only NTNU is shown and if there were several you would write NTNUs. And here is an example¹ of citation [NNNN21].

This is the second paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

And after the second paragraph follows the third paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

After this fourth paragraph, we start a new paragraph sequence. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look.

¹A footnote

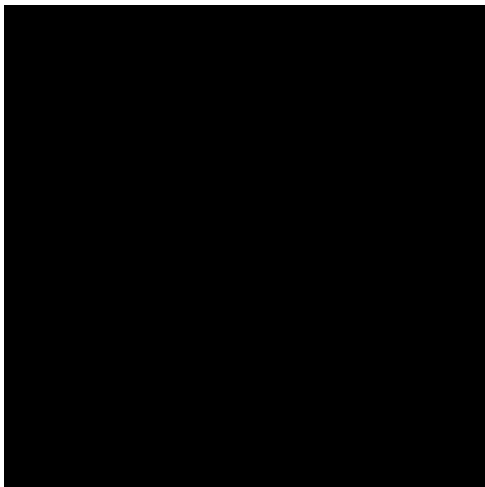


Figure 1.1: A figure

This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

1.1 First section (verbose version)

1.1.1 First subsection with some *Math* symbol

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

- item1
- item2
- ...

1.1.2 Mathematics

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. $\sin^2(\alpha) + \cos^2(\beta) = 1$. If you read this text, you will get no information $E = mc^2$. Really? Is there no information? Is there a

Table 1.1: A table

a	b	c	d	e
f	g	h	i	j
k	l	m	n	o
p	q	r	s	t
u	v	w	x	y
z	æ	ø	å	

difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. $\sqrt[n]{a} \cdot \sqrt[n]{b} = \sqrt[n]{ab}$. This text should contain all letters of the alphabet and it should be written in of the original language. $\frac{\sqrt[n]{a}}{\sqrt[n]{b}} = \sqrt[n]{\frac{a}{b}}$. There is no need for special content, but the length of words should match the language. $a \sqrt[n]{b} = \sqrt[n]{a^n b}$.

Proposition 1.1. *A proposition... (similar environments include: theorem, corollary, conjecture, lemma)*

Proof. And its proof. □

1.1.3 Source code example

Algorithm 1.1 The Hello World! program in Java.

```
class HelloWorldApp {
    public static void main(String[] args) {
        //Display the string
        System.out.println("Hello World!");
    }
}
```

You can refer to figures using the predefined command like Figure 1.1, to pages like Page 1.1, to tables like Table 1.1, to chapters like Chapter 1 and to sections like Section 1.1 and you may define similar commands to refer to proposition, algorithms etc.

Chapter 2

Introduction

The introduction must contain the following topics at the least:

- Paragraph: MPC and significance
- Garbling schemes and optimizations
- Formal Verification
- WIP TOOL Domino
- What does the thesis do? Use domino to formally verify a proof for Half Gates

Chapter 3

Background

The following sections can be split into their own chapters (if needed).

3.1 Multi-Party Computation (MPC)

3.2 Garbling Schemes

How do these relate to multi party Computation

Garbling as a cryptographic goal, as opposed to a technique to achieve some specific properties.

The properties that circuit garbling provides - privacy, authenticity and obliviousness.

General definition of a garbling scheme: [BHR12]

3.2.1 Oblivious Transfer Protocols

Why are they used in garbling schemes

3.3 Optimizations to Yao's Garbling scheme

3.3.1 Point and Permute

3.3.2 Free XOR

3.3.3 Garbled Row Reduction - 3

Compatible with Free XOR

3.3.4 Garbled Row Reduction - 2

3.3.5 FleXOR

3.3.6 Half-Gates

End with why we chose this one to work one

3.3.7 Garbled Gadgets

3.4 Formal Verification and Tools

Start from Formal verification in general (read more too), and then go into tools used for formal verification in cryptography.

3.4.1 Easycrypt

3.4.2 Domino

How is it different from easycrypt? why did we choose this? what is a state separating proof and why is this more suited to those?

Chapter 4

Pen-and-Paper State-Separating Proof

Read half gates and chris's paper to understand what to do here even.

4.1 State Separating Proofs

- Something about why state separating proofs exist. "composable" and "constructive" security

4.2 Existing Half-Gates Proof and how it is to be modelled into state separating proofs

Chapter 5

Domino

5.1 What is Domino?

What is domino and how does it work? The layers that exist, and the design in general.

Domino is a proof verifier for State-separating proofs (SSPs) that allows the user to take a pen-and-paper SSP and import it using very similar syntax and constructs such as packages, games and theorems. The explicit relation between these games is defined using invariant files, written in the SMT-LIB language. These games are then checked against different lemmata that are needed to fulfil the relation that the user wants to establish.

Domino is published as a rust package available on Github. The domino program/crate itself contains a compiler that has its own defined language that intentionally tries to be very similar the SSP syntax defined in [BDF+18].

5.2 Using Domino

Folder structure. What are Packages, Games, and Theorems (and they were called proofs before)

5.3 Coding Half-gates in Domino

5.4 Improvements

Chapter 6

Conclusion

References

- [BDE+22] C. Brzuska, A. Delignat-Lavaud, et al., “Key-schedule security for the TLS 1.3 standard”, in *Advances in Cryptology – ASIACRYPT 2022, Part I*, (Taipei, Taiwan, Dec. 5–9, 2022), S. Agrawal and D. Lin, Eds., ser. Lecture Notes in Computer Science, vol. 13791, Springer, Cham, Switzerland, 2022, pp. 621–650.
- [BDF+18] C. Brzuska, A. Delignat-Lavaud, et al., *State separation for code-based game-playing proofs*, Publication info: A major revision of an IACR publication in ASIACRYPT 2018, 2018. last visited: Apr. 10, 2026. [Online]. Available: <https://eprint.iacr.org/2018/306>
- [BHR12] M. Bellare, V. T. Hoang, and P. Rogaway, *Foundations of garbled circuits*, Publication info: Published elsewhere. Proceeding version appears in CCS 2012, 2012. last visited: Jan. 14, 2026. [Online]. Available: <https://eprint.iacr.org/2012/265>
- [BMR90] D. Beaver, S. Micali, and P. Rogaway, “The round complexity of secure protocols (extended abstract)”, in *22nd Annual ACM Symposium on Theory of Computing*, (Baltimore, MD, USA, May 14–16, 1990), ACM Press, 1990, pp. 503–513.
- [BO23] C. Brzuska and S. Oechsner, “A state-separating proof for Yao’s garbling scheme”, in *CSF 2023: IEEE 36th Computer Security Foundations Symposium*, (Dubrovnik, Croatia, Jul. 10–14, 2023), IEEE Computer Society Press, 2023, pp. 137–152.
- [BW] R. Brzuska Egger and Winkermann, private communication.
- [NNNN21] F. Name1, F. Name2, et al., “A dummy title”, *A Fake Journal*, vol. 1, no. 1, pp. 000–000, Jun. 2021.
- [Raj25] A. Rajabi, “Towards formally verifying the TLS 1.3 Key Schedule Security in SSBee”, en, *Master thesis, Aalto University, 2025*, Jul. 2025. [Online]. Available: <https://aaltodoc.aalto.fi/handle/123456789/138153>
- [Yao86] A. C.-C. Yao, “How to generate and exchange secrets (extended abstract)”, in *27th Annual Symposium on Foundations of Computer Science*, (Toronto, Ontario, Canada, Oct. 27–29, 1986), IEEE Computer Society Press, 1986, pp. 162–167.

- [ZRE15] S. Zahur, M. Rosulek, and D. Evans, “Two halves make a whole - reducing data transfer in garbled circuits using half gates”, in *Advances in Cryptology – EUROCRYPT 2015, Part II*, (Sofia, Bulgaria, Apr. 26–30, 2015), E. Oswald and M. Fischlin, Eds., ser. Lecture Notes in Computer Science, vol. 9057, Springer Berlin Heidelberg, Germany, 2015, pp. 220–250.